OptimaMT: Lighter OPUS-MT Translation Models for ONNX Runtime

Removing a duplicated output layer from 49 quantized translation models, with identical output

Amer Alreyahi
AI & PPM Solution Architect · Riyadh, Saudi Arabia
amerzuher@outlook.com · ameralreyahi.com · github.com/AmerAlreyahi
linkedin.com/in/ameralreyahi · huggingface.co/AmerAlreyahi · researchgate.net/profile/Amer-Alreyahi

ABSTRACT

OPUS-MT translation models run locally and offline through their quantized ONNX exports. ONNX Runtime decodes these exports 2.3–2.5× faster per token when the session option `session.disable_quant_qdq` is set to 1 (called *fast mode* in this report), but in that mode the models also need about twice the memory. We trace the extra memory to one detail of the exported decoder: the subgraph that builds the output layer (a Transpose followed by a DequantizeLinear of the shared 8-bit embedding) is present in both branches of the decoder's If node, and ONNX Runtime folds it to a float matrix once per branch. Moving the two nodes in front of the If, without changing any weight, removes the second copy. We applied this change to 49 translation directions and measured them on Linux and Windows with ONNX Runtime 1.31.1. In fast mode, memory after loading fell from a median of 525 to 281 MiB on Linux and from 500 to 257 MiB on Windows (45–49 % and 47–51 % per pair). Decoding speed did not change. On FLORES-200 devtest all 49 pairs produce the same tokens as the original files under both session settings, so BLEU and chrF are identical as well. The models are published as drop-in replacements.

Keywords machine translation, OPUS-MT, MarianMT, ONNX Runtime, quantization, memory footprint, on-device inference, transformers.js

MODELS
huggingface.co/OptimaMT

CODE AND DATA
github.com/AmerAlreyahi/OptimaMT

RESULTS AND DEMO
huggingface.co/spaces/OptimaMT/results

How to cite Alreyahi, A. (2026). OptimaMT: Lighter OPUS-MT Translation Models for ONNX Runtime. Technical Report v1.0. https://github.com/AmerAlreyahi/OptimaMT

Text CC BY 4.0 · model files CC BY 4.0 · code MIT. No endorsement by the authors of OPUS-MT, the ONNX exports or ONNX Runtime is implied.

Contents

1 **Machine translation in one page** 3

2 **Inside a translation model** 4
- 2.1 Tokens and SentencePiece 4
- 2.2 Embeddings 5
- 2.3 The Transformer encoder and decoder 5
- 2.4 Attention in plain words 6
- 2.5 The output layer 6
- 2.6 Greedy decoding, step by step 7

3 **From PyTorch to ONNX** 7
- 3.1 ONNX and ONNX Runtime 7
- 3.2 The export and the merged decoder 8
- 3.3 8-bit quantization 9

4 **The problem: fast mode doubles the memory** 9
- 4.1 ONNX Runtime's fast mode 9
- 4.2 The output layer is folded twice 9

5 **The fix** 11
- 5.1 What the rebuild does 11
- 5.2 Why the output is identical 12
- 5.3 A known pattern 12

6 **Results** 12
- 6.1 Measurement setup 12
- 6.2 Memory on the reference pairs 12
- 6.3 Speed 14
- 6.4 All 49 pairs 14
- 6.5 Identical output on FLORES-200 16
- 6.6 ONNX Runtime versions 17
- 6.7 In the browser 18
- 6.8 Drop-in check 19

7 **Using OptimaMT** 19
- 7.1 JavaScript (transformers.js) 19
- 7.2 Python (ONNX Runtime, no PyTorch) 19
- 7.3 Available pairs 20
- 7.4 Used in production 20

8 **Limitations** 20
- 8.1 Future work 20

9 **Reproducibility, licences and acknowledgements** 21
- 9.1 Reproducing the results 21
- 9.2 Licences 21
- 9.3 Acknowledgements 21

References 22

Appendix A **All 49 pairs** 23

Appendix B **Glossary** 24

About the author 25

Key results

Table 1. Key results. ONNX Runtime 1.31.1, CPU, 4 threads. Fast mode means `session.disable_quant_qdq = 1`.

Measure	Linux	Windows
Fast mode memory after load, median of 49 pairs	525 → 281 MiB	500 → 257 MiB
Fast mode saving per pair	45–49 %	47–51 %
Default settings memory, median of 49 pairs	277 → 217 MiB	196 → 166 MiB
Decoder time per token, defaults → fast mode (median, 5 pairs)	17.3 → 6.9 ms	21.3 → 9.1 ms
Speed change caused by the rebuild (fast mode)	none (6.9 → 6.9 ms)	none (9.2 → 9.1 ms)
FLORES-200 devtest, identical output	98 of 98 runs	token ids match Linux

The 2.3–2.5× speed-up is ONNX Runtime's fast mode compared with its defaults and is not a result of this work. What OptimaMT changes is the memory cost of that mode.

All numbers in this report come from the measurement files in the repository's results folder; Section 9.1 lists the scripts that produce them.

1 Machine translation in one page

Machine translation (MT) is software that turns text in one language into text in another. Current MT systems are neural networks: they learn from large collections of example sentence pairs and then translate new sentences without hand-written rules.

Most people use MT through a cloud service. The text is sent to a remote server, translated there and sent back. This works well, but it has three costs. The text leaves the user's device, which matters for private documents, medical or legal material and anything under a confidentiality agreement. Each request costs money or is limited by a quota. And nothing works without a network connection.

Local MT runs the model on the user's own computer, or inside a web page in the browser. Nothing is sent anywhere, there is no per-request cost, and it works offline. The price is that the user's machine has to do the work. A local model must therefore be small enough to load quickly and fast enough to feel instant, on ordinary laptops as well as on developer workstations.

OPUS-MT, from the Language Technology Research Group at the University of Helsinki, publishes open translation models for many language directions, each a separate model [1]. The 49 directions studied here have 72 to 85 million parameters each, depending on their vocabulary size. Quantized ONNX versions of these models (Section 3) let them run inside desktop applications and, with transformers.js [9], in the browser.

This report is about one setting for those ONNX versions: the ONNX Runtime option that makes them 2.3–2.5× faster, and why that option doubled their memory use. The fix described in Section 5 keeps the speed and removes the extra memory. The output stays the same, token for token.

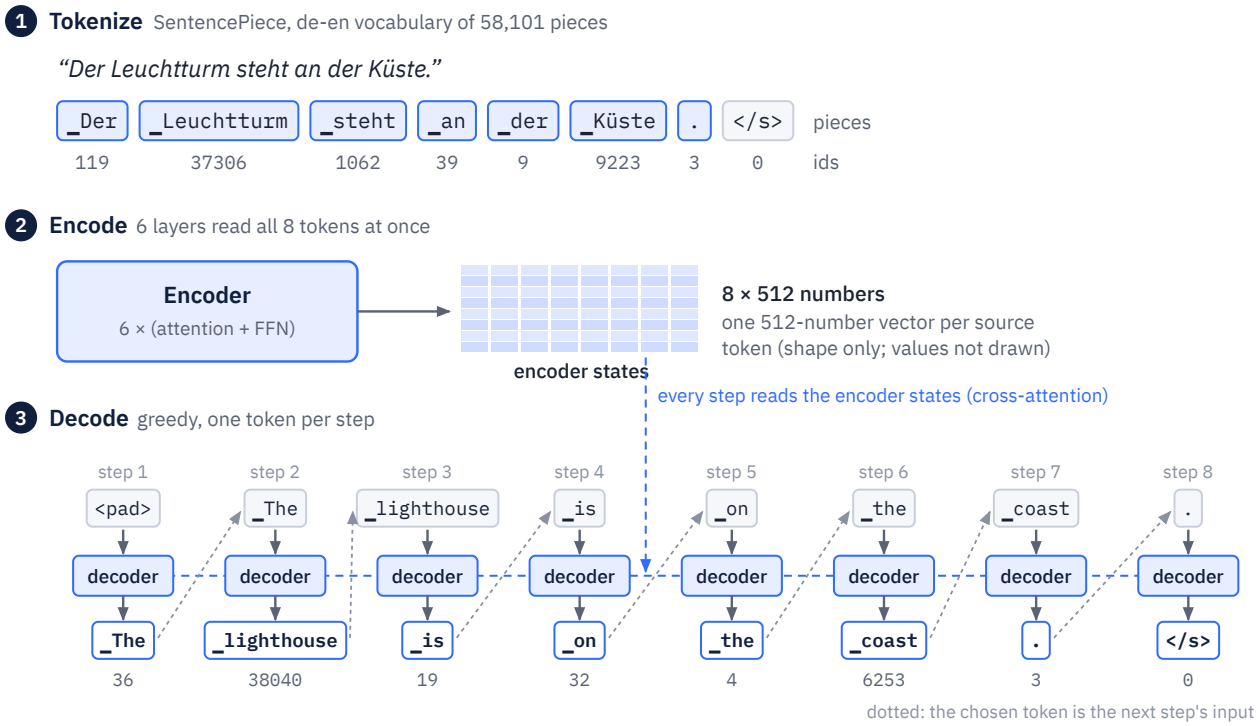


Figure 1. The translation pipeline for the example used throughout this report (OPUS-MT de-en). Token pieces, ids and the chosen tokens are the real values from the model's tokenizer and its greedy decoding; the output is the model's actual translation.

2 Inside a translation model

This chapter explains how an OPUS-MT model turns a sentence into a translation. It uses one German sentence and the real de-en model throughout, so every number shown can be reproduced.

2.1 Tokens and SentencePiece

A neural network works with numbers, not letters. The first step is to cut the text into *tokens*: whole words for common words, pieces of words for rarer ones. OPUS-MT uses SentencePiece [4] for this. SentencePiece learns its list of pieces from the training data, so frequent words get a single piece and unusual words are spelled out from smaller pieces. A leading `_` marks a piece that starts a new word.

The sentence *Der Leuchtturm steht an der Küste.* becomes seven pieces: `_Der`, `_Leuchtturm`, `_steht`, `_an`, `_der`, `_Küste` and `.` (the full stop). Each piece is looked up in the model's vocabulary, a fixed list of 58,101 entries for de-en, giving the ids `119 37306 1062 39 9 9223 3 0`. The final `0` is the end-of-sentence token `</s>`. The vocabulary size differs per language pair, from 53,685 to 80,379 entries in the 49 pairs studied here.

2.2 Embeddings

Each token id is then replaced by a vector of 512 numbers, its *embedding*. The embeddings are part of the trained model: a table with one row of 512 numbers per vocabulary entry. During training, words used in similar contexts tend to get similar vectors. The position of each token in the sentence is added to its vector (a fixed sinusoidal pattern in these models), so the model knows the word order.

In OPUS-MT the same embedding table is shared by the encoder input, the decoder input and the output layer (Section 2.5). This sharing is what makes the output layer large and is central to the rest of this report.

2.3 The Transformer encoder and decoder

OPUS-MT models use the MarianMT architecture [2], a Transformer [3] with two halves. The **encoder** reads the whole source sentence at once and produces one 512-number vector per source token, now informed by the surrounding words. The **decoder** writes the translation one token at a time. At each step it looks at the tokens it has written so far and at the encoder's output, and it produces a score for every entry in the vocabulary (Figure 2).

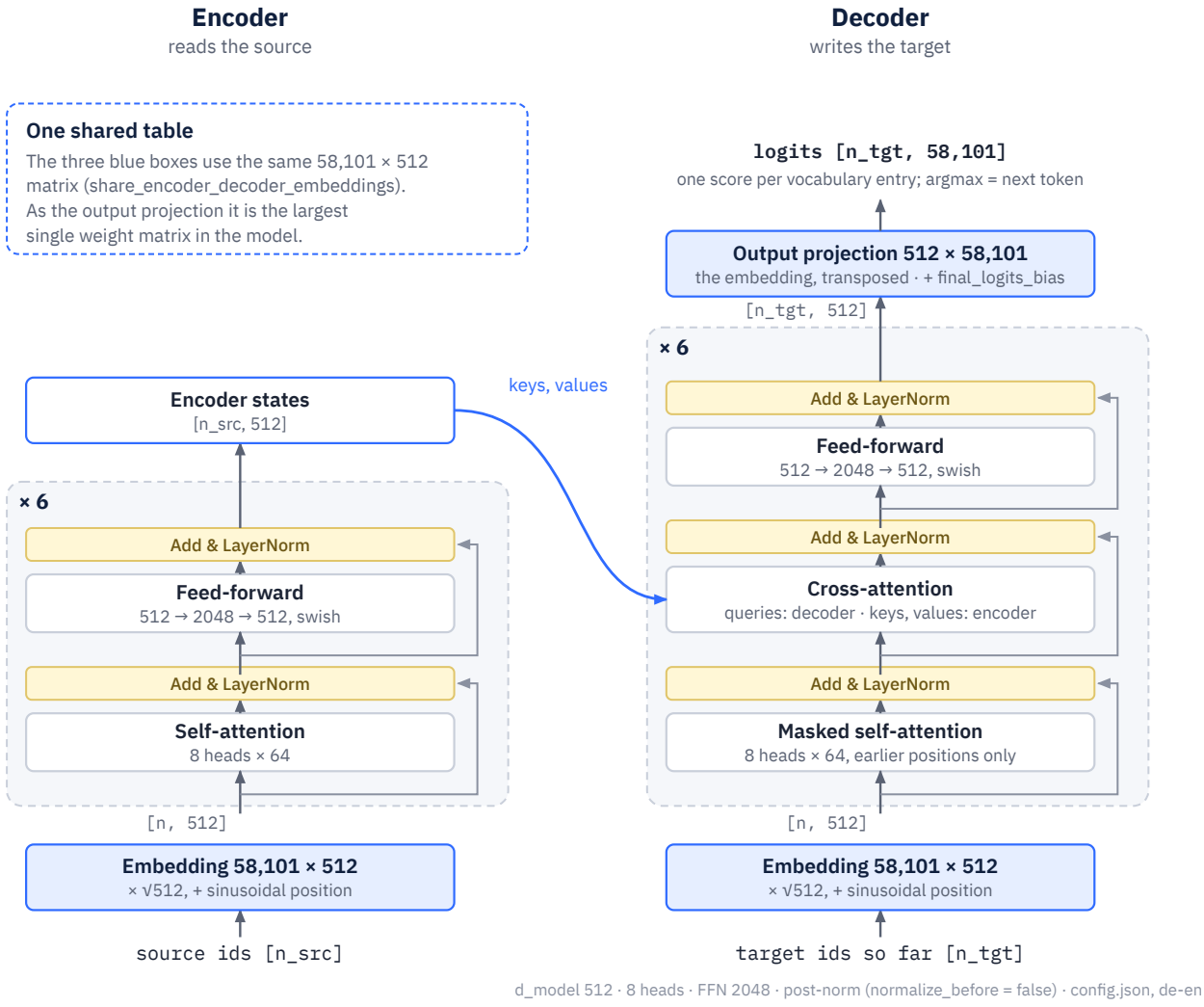


Figure 2. The MarianMT encoder-decoder used by all 49 pairs, drawn with the de-en sizes. Each half repeats its block six times; every sublayer is followed by a residual addition and layer normalization (post-norm). The three blue boxes are one shared weight matrix. Sizes from the model's config.json; shapes from the ONNX graph.

Table 2. Model dimensions, identical in all 49 pairs (from each pair's config.json).

Setting	Value	Meaning
encoder_layers / decoder_layers	6 / 6	blocks stacked in each half
d_model	512	numbers per token vector
attention heads	8	parallel attention patterns per layer (64 numbers each)
feed-forward size	2048	width of the inner layer of each block
activation	swish	non-linearity in the feed-forward layer
shared embeddings	yes	one table for encoder, decoder and output layer
vocabulary	53,685–80,379	differs per pair

2.4 Attention in plain words

Attention is the mechanism that lets each token gather information from other tokens. For every token the layer computes how relevant every other token is (a set of weights that add up to 1) and then takes a weighted average of their vectors. *Self-attention* does this within one sentence. *Cross-attention*, in the decoder, lets each output position look at the source sentence. Each layer runs eight of these patterns in parallel (the heads), so it can follow several relationships at once.

Figure 3 shows real cross-attention weights from the de-en model for the example sentence. The values were read from the decoder's fifth layer and averaged over its eight heads. When the decoder produces *lighthouse* it attends mostly to *Leuchtturm* (0.67); for *coast* it attends to *Küste* (0.74). One position earlier, while producing *the*, it already attends to *Küste* (0.50): the decoder looks at the noun it is about to translate. Much of the remaining weight goes to the end-of-sentence token `</s>`.

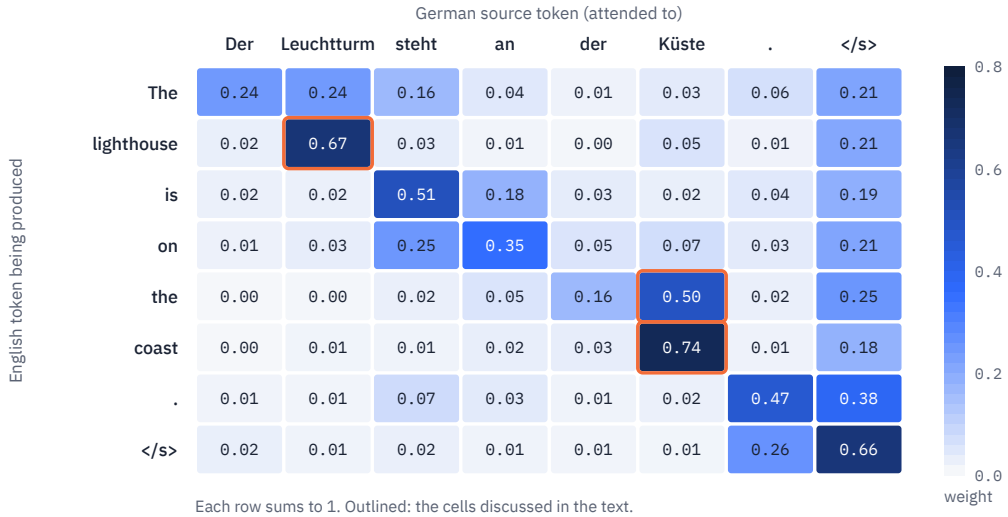


Figure 3. Cross-attention in the de-en model: decoder layer 5 of 6, mean of 8 heads. Each row is the English token being produced, each column a German source token. Measured by running the model's own ONNX decoder with the attention weights exposed as extra outputs.

2.5 The output layer

After the last decoder layer, each position is a vector of 512 numbers. The **output layer**, also called the output projection, turns that vector into one score per vocabulary entry: a matrix multiplication with the embedding table, transposed. For en-ar, with a vocabulary of 62,802, this matrix has 512 × 62,802 = 32.2 mill-

ion entries, about 42 % of the model's 76 million parameters. In 32-bit floating point it occupies 122.7 MiB. This one matrix is what the rest of the report is about.

2.6 Greedy decoding, step by step

The decoder starts from a start token (for OPUS-MT, the padding token, id 58,100 in de-en) and runs once per output token. At each step it scores all 58,101 vocabulary entries; *greedy decoding* takes the highest score. The chosen token is appended and fed back in, and the loop stops when the model chooses `</s>`. Figure 4 shows the real steps for the example, with the probabilities of the three best candidates.



Figure 4. Greedy decoding of *Der Leuchtturm steht an der Küste.* with the de-en model: at each step the token fed in, the three highest-scoring candidates and their probabilities (softmax of the decoder's scores, padding excluded). The top candidate is chosen every time.

The result is *The lighthouse is on the coast.* At step 3 the model weighed *is* (0.54) against *stands* (0.21), the literal translation of *steht*. Beam search, which keeps several candidates open, can pick a different sentence; all results in this report use greedy decoding, as the measurement scripts do.

> **The key/value cache**
>
> At each step the decoder needs the attention keys and values of every earlier position. Recomputing them would make each step slower than the last. Instead, each step returns them as outputs (`present.*`) and the next step receives them as inputs (`past_key_values.*`): 24 tensors for 6 layers, each of shape [batch, 8, length, 64]. The cross-attention keys and values for the source are computed once, on the first step, and reused. This is why the first step and the later steps run different code, which leads directly to the If node in Section 3.2.

3 From PyTorch to ONNX

3.1 ONNX and ONNX Runtime

OPUS-MT models are trained with the Marian toolkit and published as PyTorch checkpoints on Hugging Face. Running PyTorch inside a desktop application or a web page is impractical: the library is large and tied to Python. **ONNX** (Open Neural Network Exchange) [7] is a file format that describes a model as a graph of standard operations (MatMul, Softmax, Transpose and so on) plus its weights. **ONNX Runtime** [6], from Microsoft, is an engine that loads such a graph and runs it on CPUs and GPUs, from C, C++, C#, Python, Rust or JavaScript. onnxruntime-web, its WebAssembly build, runs the same files in a browser.

When ONNX Runtime loads a model it first optimizes the graph. One of these optimizations is *constant folding*: any operation whose inputs are all constants (weights) is computed once at load time, and its result is stored in place of the operation. This saves work at every run but costs memory for the stored result.

3.2 The export and the merged decoder

The ONNX files used here come from Xenova's exports on Hugging Face (44 pairs), made with Hugging Face Optimum [8] for transformers.js [9]. Five further pairs (fr-ar, es-ar, ar-fr, ar-es, ar-de) were exported from the Helsinki-NLP weights with Optimum in the same layout. Each pair has an encoder file and a *merged* decoder file.

The decoder has to handle two cases: the first step, with no cache, and every later step, with the cache from Section 2.6. Optimum stores both versions in one file and selects between them with an ONNX **If** node, a conditional: the input `use_cache_branch` is false on the first step and true afterwards. The `no_past` branch holds the first-step graph (1,589 nodes in en-ar), the `with_past` branch the later-step graph (1,333 nodes). Both branches contain the complete decoder, including the output layer (Figure 5). The weights themselves are stored once, in the main graph, and both branches refer to them.

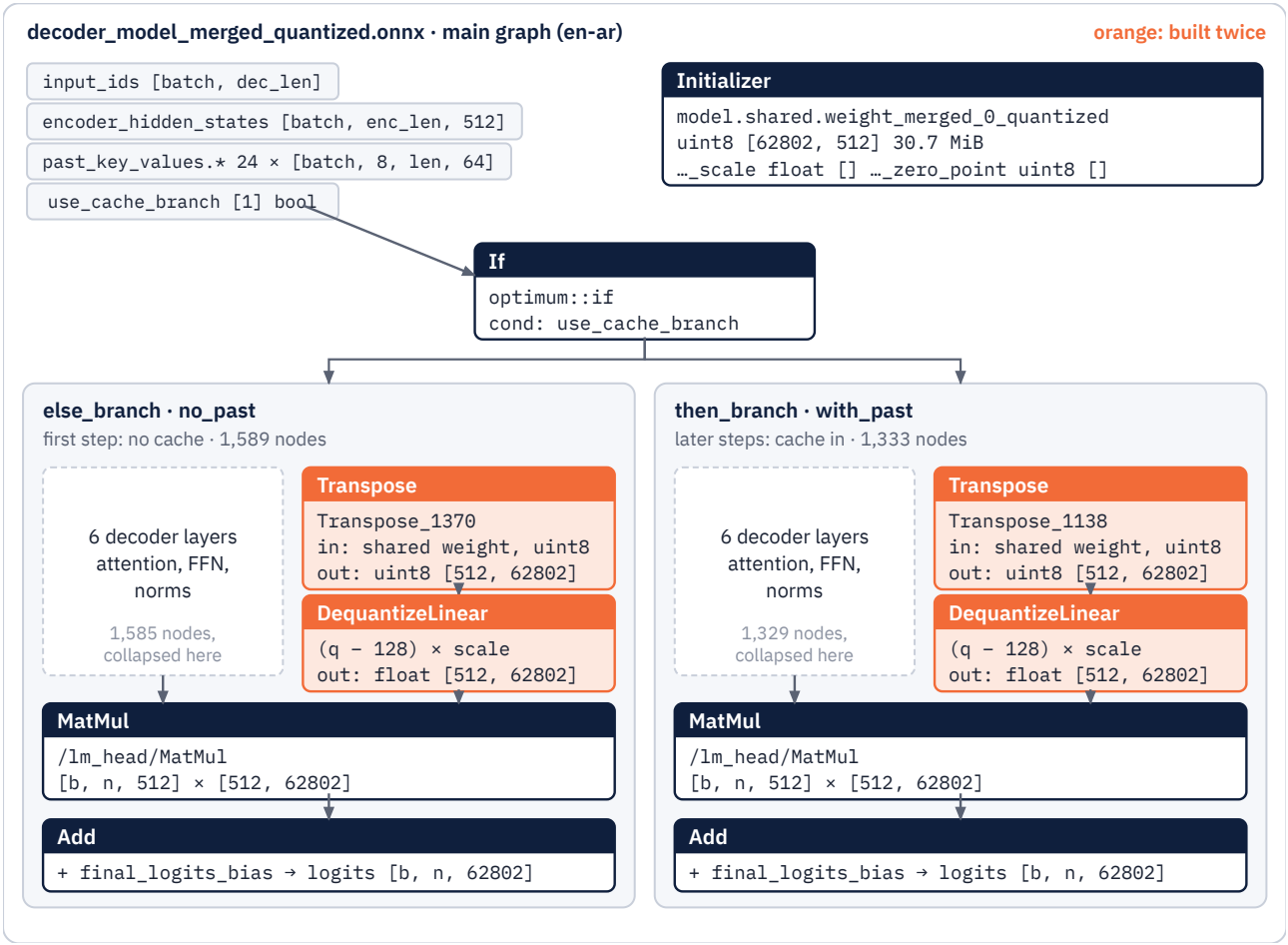
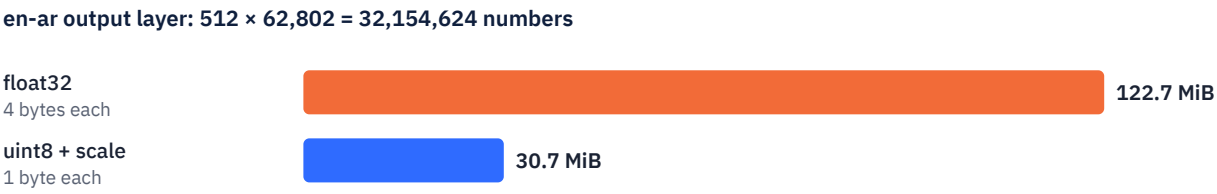


Figure 5. The merged en-ar decoder as stored in the file, with the real node names and shapes. One If node chooses between two complete decoder graphs. Both build the same output-layer subgraph (orange) from the same stored weight.

3.3 8-bit quantization

To make the files smaller, the exports are *quantized*: most weights are stored as 8-bit integers instead of 32-bit floats, a quarter of the size. Each quantized tensor carries a scale and a zero point, and a **DequantizeLinear** node converts it back to float when it is used: float = (integer − zero point) × scale. In these files the shared embedding is stored as unsigned 8-bit integers (`uint8`) with one scale and one zero point for the whole tensor. The output layer reads it through a Transpose (to put the matrix in the right orientation for the multiplication) and a DequantizeLinear (Figure 6).




```
DequantizeLinear: float = (q − zero_point) × scale
```
in en-ar: one scale for the whole tensor, 0.0067904945, and zero_point 128

Figure 6. What 8-bit storage saves for the en-ar output layer. Sizes follow from the tensor shape; scale and zero point are read from the model file.

This dynamic quantization is lossy, but the loss happens once, when the file is made. Everything in this report compares two files with exactly the same 8-bit weights, so the quantization error is the same in both.

4 The problem: fast mode doubles the memory

4.1 ONNX Runtime's fast mode

By default ONNX Runtime keeps a DequantizeLinear that sits on a weight. Its QDQ optimizations look for these nodes to recognise quantized patterns, so its constant folding leaves them alone on purpose. The session option `session.disable_quant_qdq = 1` turns that handling off. Constant DequantizeLinear nodes are then folded like any other constant operation: the weights are converted to float once, at load time, and the multiplications that use them run in float.

On these models that is much faster. Measured on five reference pairs, the decoder takes 17.3 ms per token with the defaults and 6.9 ms in fast mode on Linux, and 21.3 versus 9.2 ms on Windows (Figure 10 in Section 6.3). We did not profile which operators account for the difference. The cost is memory: float weights take four times the space of 8-bit ones, and the original files needed 525 MiB after loading in fast mode on Linux, against 277 MiB with the defaults (medians of 49 pairs).

4.2 The output layer is folded twice

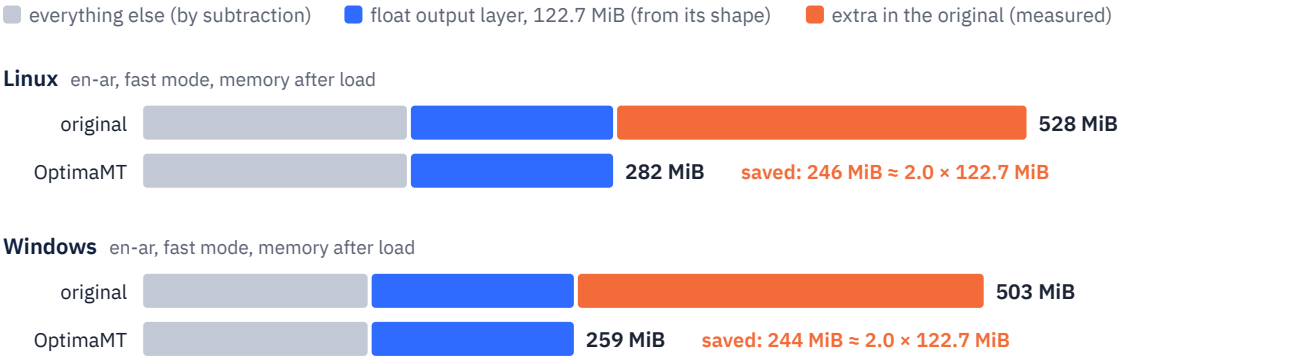
Some growth is expected, since float weights are larger. But the measured growth is larger than the weights explain. ONNX Runtime can write the optimized graph it actually runs to a file (`SessionOptions.optimized_model_filepath`), which shows what is kept after folding. Table 3 lists the large tensors for en-ar.

Table 3. Large tensors kept by ONNX Runtime 1.31.1 after its load-time optimizations, en-ar decoder (scripts/inspect_graph.py, results/graph.json).

Decoder	Session settings	Where	Tensor kept	Size
Original	defaults	main	8-bit embedding	30.7 MiB
		no_past	8-bit, transposed	30.7 MiB
		with_past	8-bit, transposed	30.7 MiB
Original	fast mode	main	8-bit embedding	30.7 MiB
		no_past	**float, transposed**	**122.7 MiB**
		with_past	**float, transposed**	**122.7 MiB**
OptimaMT	defaults	main	8-bit embedding	30.7 MiB
		main	8-bit, transposed	30.7 MiB
OptimaMT	fast mode	main	8-bit embedding	30.7 MiB
		main	float, transposed	122.7 MiB

ONNX Runtime folds constants inside each subgraph separately. Each branch of the If holds its own copy of Transpose → DequantizeLinear on the shared embedding, so each branch gets its own folded result. In fast mode that is a 122.7 MiB float matrix (62,802 × 512 × 4 bytes) per branch: the output layer exists twice in memory. With the defaults the same happens on a smaller scale: the transposed 8-bit matrix (30.7 MiB) is stored once per branch.

Figure 7 puts this next to the measured totals for en-ar. With ONNX Runtime 1.31.1 the memory saved by removing the duplicate is about twice the size of the folded matrix (246 MiB on Linux, 244 MiB on Windows). With ONNX Runtime 1.28 it is about one matrix (Section 6.6). We did not trace which internal buffers ONNX Runtime keeps in each version, so the figure shows the extra part of the original's memory as measured, not as a breakdown.



Totals measured (results/reference*.csv); the blue part follows from the tensor shape 512 × 62,802 × 4 bytes.

Figure 7. Where the memory goes, en-ar in fast mode. The totals are measured; the blue part is the 122.7 MiB float matrix, computed from its shape; the orange part is the measured difference between the original and the rebuilt decoder.

> **Not pre-packing**
>
> This is constant folding of a duplicated subgraph. It is not related to `MatMulInteger` weight pre-packing or any other kernel-level feature. No weight was changed, retrained or re-quantized.

5 The fix

5.1 What the rebuild does

Both branches compute the same value from the same stored weights. The fix computes it once, outside the branches. The script `convert.py` does the following for each pair:

1. Load the merged decoder and find its single If node and the two branch graphs.
2. In each branch, find the output layer: the one DequantizeLinear whose input comes from a Transpose.
3. Check that every input of these two nodes is a constant stored in the main graph or in the branch.
4. Move the Transpose and the DequantizeLinear into the main graph, directly in front of the If, together with any constant they read that lived inside a branch.
5. For the second branch, check that its two nodes are identical to the ones already moved, and that any constants they read are byte-for-byte equal. Then delete them. Both branches now read the one outer value.
6. Run `onnx.checker` on the result and save it.

Nothing else changes: same inputs and outputs, same 8-bit weights, same scales and zero points, same operator versions. In en-ar the main graph goes from 2 to 4 nodes and each branch loses 2 (1,589 → 1,587 and 1,333 → 1,331); the file is 332 bytes smaller. The script is deterministic: running it on the original Xenova file reproduces the published file byte for byte, and it checks the result against the SHA-256 recorded in the repository.

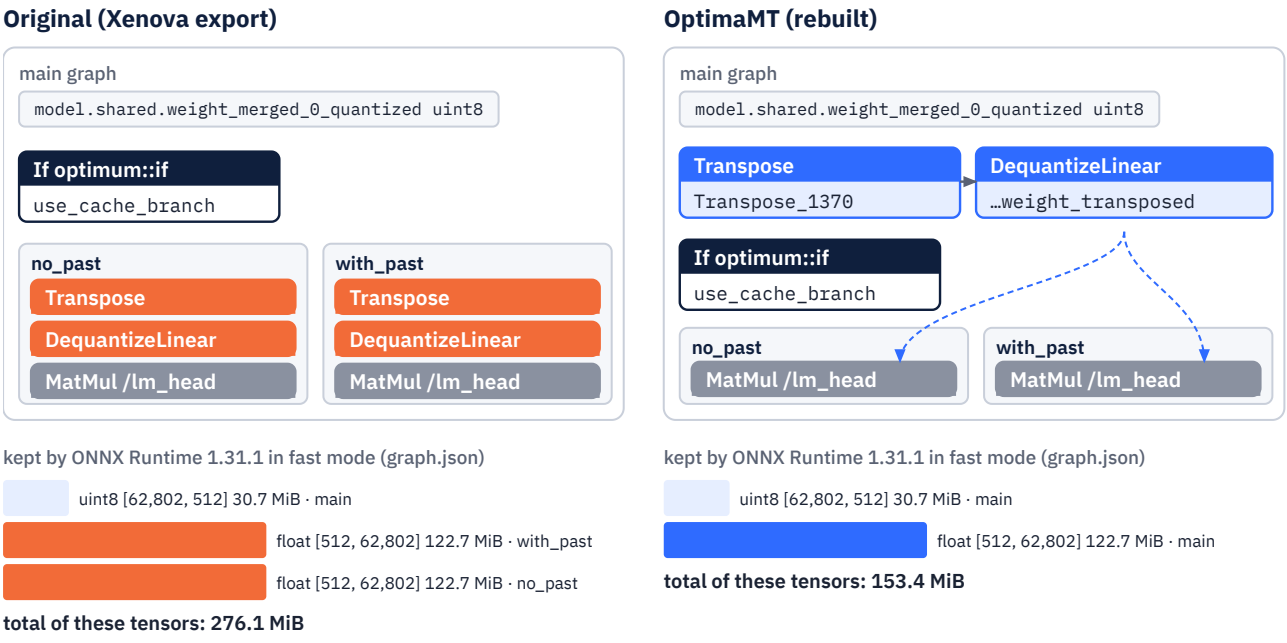


Figure 8. The en-ar decoder before and after the rebuild, and the large tensors ONNX Runtime keeps from it in fast mode (bars to scale). Left: each branch has its own Transpose and DequantizeLinear, so the output layer is folded twice. Right: the two nodes sit in the main graph and are folded once; both branches read the result.

5.2 Why the output is identical

The moved nodes compute the same function of the same constants as before. Inside each branch, the MatMul that produces the logits receives a tensor with the same values, shape and type as before; only the place where it was computed has moved. The rebuild itself does not reorder, fuse or approximate any operation. The decoder therefore produces the same logits and greedy decoding chooses the same tokens. Section 6.5 confirms this on the full FLORES-200 devtest set.

5.3 A known pattern

Moving a computation that both sides of a conditional share out of the conditional is a standard compiler transformation, and ONNX Runtime's per-subgraph constant folding is known behaviour. The work reported here applies this to the 49 published OPUS-MT exports, verifies that the output does not change, and measures the effect on two operating systems and in the browser. The same change could be made in the exporter, which would make the rebuild unnecessary for new exports.

6 Results

6.1 Measurement setup

Table 4. Machines and software (results/environment*.json).

	Linux	Windows
CPU	Intel(R) Core(TM) Ultra 7 265KF	virtual machine on the same CPU
Logical CPUs / RAM	20 / 31.1 GB	4 / 8.0 GB
Operating system	Linux 6.6 (WSL2), glibc 2.39	Windows 10 (build 19045)
Python / ONNX Runtime	3.12.3 / 1.31.1	3.12.12 / 1.31.1
Threads	4 intra-op, 1 inter-op	4 intra-op, 1 inter-op
Memory measure	RSS	working set; peaks sampled every 2 ms

- **After load:** resident memory after creating the encoder and decoder sessions, minus the value just before.
- **Peak, 20 lines:** the highest resident memory while translating a fixed batch of 20 sentences (on Linux `VmHWM`, reset through `/proc/self/clear_refs`).
- **ms per token:** decoder time per generated position, median of 5 runs after a warm-up.
- Each configuration runs in a fresh process. *Original* is the Xenova (or Helsinki-NLP) file; *OptimaMT* is the rebuilt file. All other files are the same.

6.2 Memory on the reference pairs

Five pairs (en-ar, ar-en, en-de, de-en, ja-en) were measured in full detail. Figure 9 shows memory after load; Table 5 and Table 6 give every value. In fast mode the rebuild saves 46–47 % on Linux and 48–49 % on Windows. With the defaults it saves 21–22 % on Linux and 15–17 % on Windows.

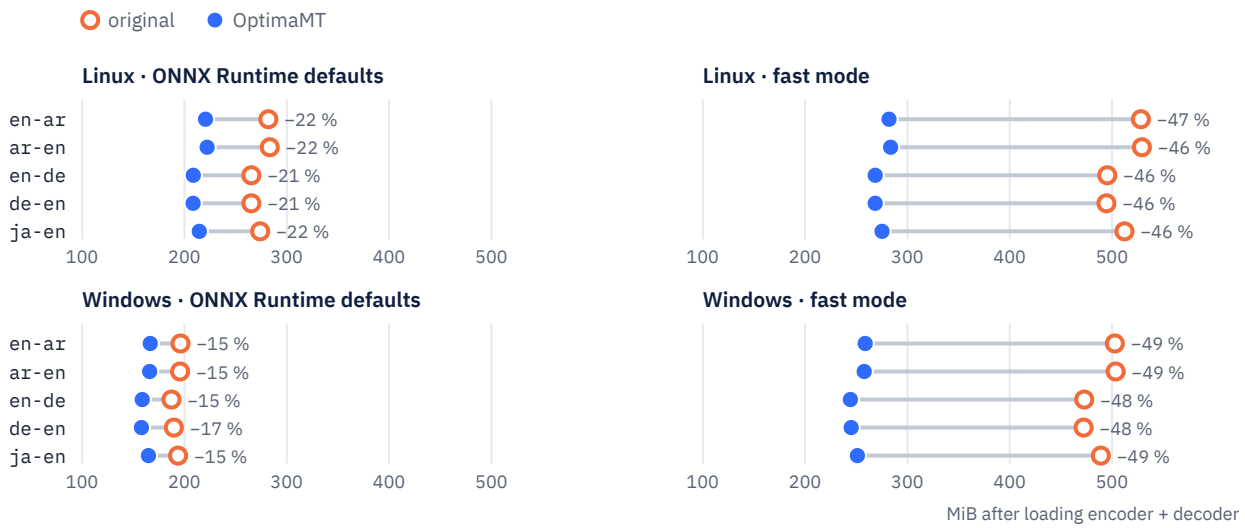


Figure 9. Memory after loading encoder and decoder, original against OptimaMT, both session settings, Linux and Windows, five reference pairs.

Table 5. Linux, reference pairs. Original → OptimaMT (results/reference.csv).

Pair	Settings	After load (MiB)	Change	Peak, 20 lines (MiB)	ms / token	Same tokens
en-ar	defaults	282 → 220	−22 %	529 → 476	17.2 → 17.3	yes
	fast mode	528 → 282	−47 %	605 → 358	7.2 → 6.8	yes
ar-en	defaults	283 → 222	−22 %	530 → 479	18.7 → 17.2	yes
	fast mode	529 → 284	−46 %	595 → 349	7.8 → 7.1	yes
en-de	defaults	265 → 208	−21 %	520 → 445	17.4 → 17.4	yes
	fast mode	495 → 268	−46 %	561 → 333	6.8 → 6.9	yes
de-en	defaults	265 → 208	−21 %	520 → 437	17.3 → 15.1	yes
	fast mode	494 → 268	−46 %	547 → 321	6.9 → 6.9	yes
ja-en	defaults	274 → 214	−22 %	531 → 464	15.7 → 18.3	yes
	fast mode	512 → 275	−46 %	579 → 342	6.7 → 7.4	yes

Table 6. Windows, reference pairs. Original → OptimaMT (results/reference_windows.csv).

Pair	Settings	After load (MiB)	Change	Peak, 20 lines (MiB)	ms / token	Same tokens
en-ar	defaults	196 → 166	−15 %	397 → 368	22.1 → 21.2	yes
	fast mode	503 → 259	−49 %	583 → 339	9.8 → 10.2	yes
ar-en	defaults	196 → 166	−15 %	390 → 365	22.9 → 22.6	yes
	fast mode	504 → 258	−49 %	573 → 327	9.8 → 10.6	yes
en-de	defaults	187 → 159	−15 %	469 → 346	21.3 → 19.3	yes
	fast mode	473 → 244	−48 %	541 → 313	8.8 → 8.5	yes
de-en	defaults	190 → 158	−17 %	466 → 338	20.9 → 19.9	yes
	fast mode	472 → 245	−48 %	527 → 300	9.2 → 9.1	yes
ja-en	defaults	194 → 165	−15 %	483 → 363	21.3 → 20.2	yes
	fast mode	489 → 251	−49 %	560 → 322	8.8 → 8.9	yes

The peak while translating drops as well: in fast mode on Linux from a median of 579 to 342 MiB (–41 %). The short peak during loading moves less (630 → 511 MiB median), because folding briefly holds intermediate copies in both versions.

6.3 Speed

The rebuild does not change decoding speed. In fast mode the median over the five pairs is 6.9 ms per token before and 6.9 ms after on Linux, 9.2 and 9.1 ms on Windows. Individual pairs move by a few tenths of a millisecond in both directions. Figure 10 shows the larger effect, which belongs to ONNX Runtime: fast mode is 2.5× faster than the defaults on Linux and 2.3× on Windows. With OptimaMT that speed no longer costs extra memory.

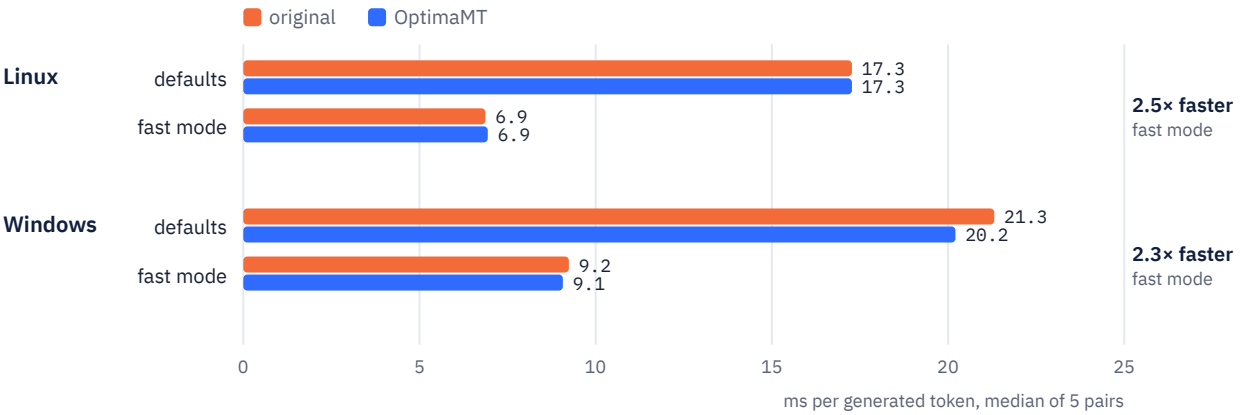


Figure 10. Decoder time per generated token, median of the five reference pairs. Fast mode is ONNX Runtime's speed-up; the rebuild changes memory, not speed.

6.4 All 49 pairs

Every pair was measured after load in both settings on both systems (Figure 11). In fast mode the saving is 45–49 % per pair on Linux and 47–51 % on Windows. With the defaults it is 14–22 % on Linux and 14–17 % on Windows. The pairs with a smaller saving in the Linux defaults panel are the six with the largest vocabularies (it-en, en-it, es-fr, fr-es, nl-en, en-nl); we did not investigate why their saving with the defaults is smaller. Appendix A lists every pair.

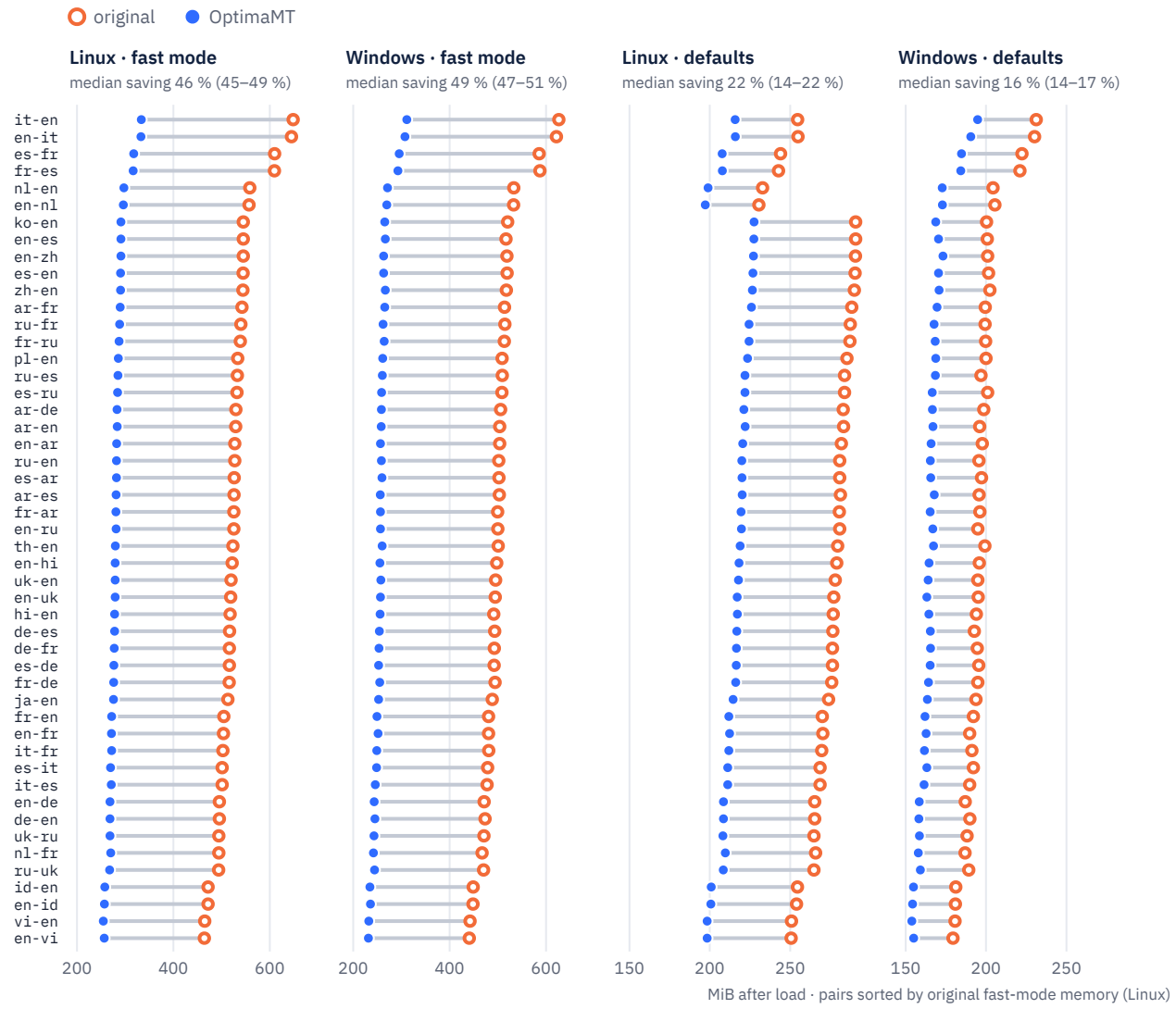


Figure 11. Memory after load for every pair, original against OptimaMT, for each system and setting. Each panel has its own scale; the median saving is given above it.

Because the duplicated matrix is the output layer, its size grows with the vocabulary. Figure 12 plots fast mode memory against vocabulary size. A least-squares line through the 49 pairs rises by 6.9 MiB per 1,000 vocabulary entries for the original files and by 3.0 MiB for OptimaMT. The difference, 3.9 MiB, is about twice the float size of 1,000 rows of the output matrix (512 × 1,000 × 4 bytes = 1.95 MiB).

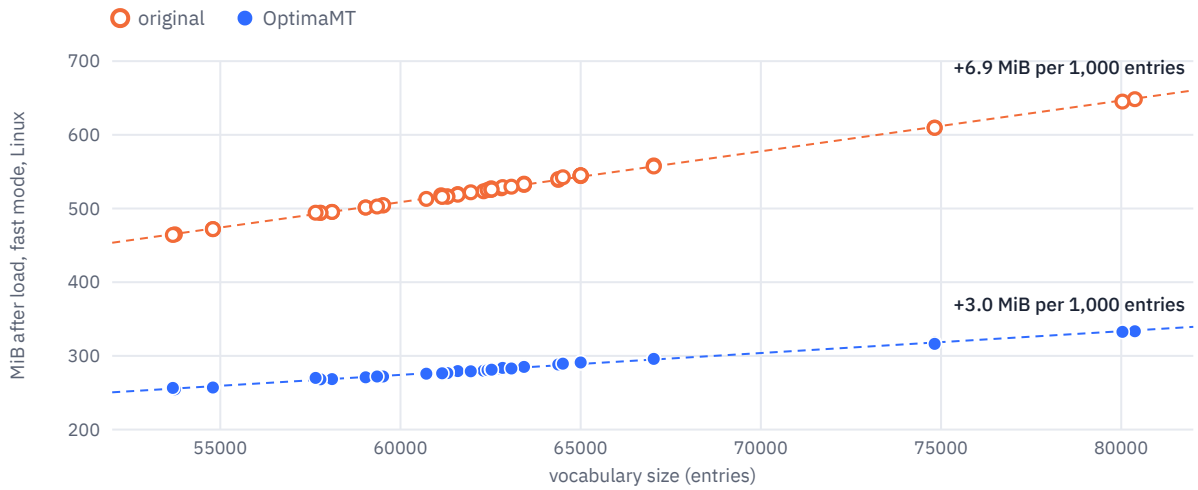


Figure 12. Fast mode memory after load against vocabulary size, all 49 pairs, Linux, with least-squares lines.

6.5 Identical output on FLORES-200

FLORES-200 [5] is a multilingual test set with the same 1,012 sentences translated into 200 languages. Every pair was run on the full devtest split, greedy, in batches of 16, with both session settings: 49 pairs × 2 settings × 1,012 sentences. The original and rebuilt decoders produced identical token ids in 98 of 98 runs, so BLEU [12] and chrF [11], computed with sacrebleu 2.5.1 [10], are identical too (Figure 13). BLEU ranges from 10.52 (es-ar) to 46.42 (en-fr). These are the original models' greedy scores and are given for reference, not as a quality claim.

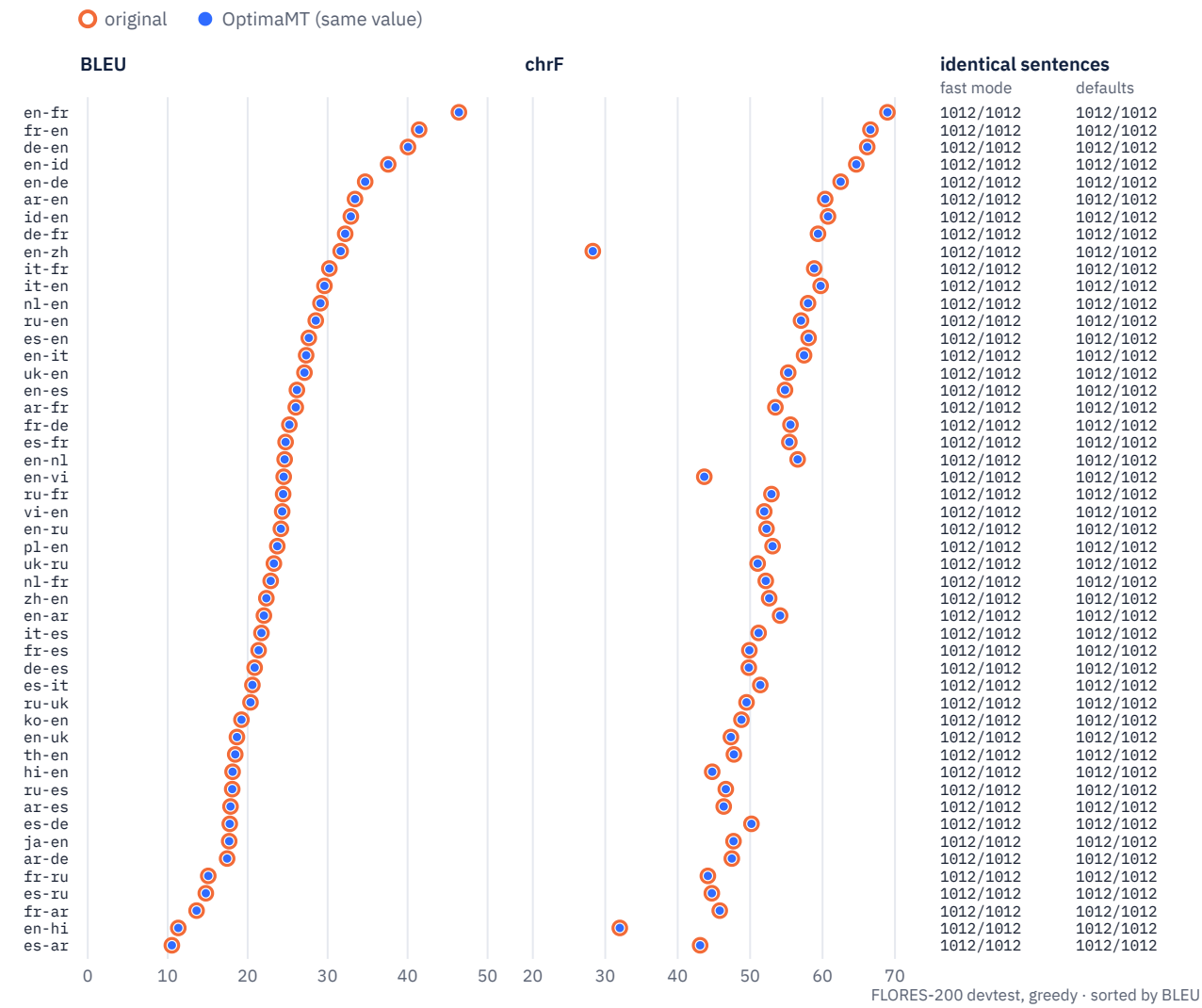


Figure 13. FLORES-200 devtest, all 49 pairs: BLEU and chrF of the original (ring) and OptimaMT (dot) in fast mode, and the number of sentences with identical tokens in both settings. The two markers sit on the same value in every row.

A smaller check, 20 fixed sentences per pair with both settings, also gave identical tokens in 98 of 98 runs. The token ids of the reference batch are the same on Windows and Linux.

6.6 ONNX Runtime versions

Table 7. ONNX Runtime 1.28.0 and 1.31.1, Linux, median of the five reference pairs, memory after load (results/reference_ort1.28.csv, results/reference.csv).

ONNX Runtime	Settings	Original (MiB)	OptimaMT (MiB)	Change	ms / token, OptimaMT
1.28.0	defaults	275	216	-59 (−22 %)	16.3
	fast mode	394	275	-119 (−30 %)	7.1
1.31.1	defaults	274	214	-59 (−22 %)	17.3
	fast mode	512	275	-237 (−46 %)	6.9

With ONNX Runtime 1.31.1 the memory saved in fast mode is about twice the folded matrix; with 1.28.0 about once. In fast mode the rebuilt decoder needs the same memory in both versions (275 MiB); the original needed 394 MiB with 1.28.0 and 512 MiB with 1.31.1. Tokens were identical between original and OptimaMT in both versions.

6.7 In the browser

transformers.js 3.8.1 runs the same files in a browser through onnxruntime-web (WebAssembly, 4 threads). It was measured in headless Chromium 156, one fresh browser per configuration. Memory is the resident memory of all browser processes minus that of a blank page.

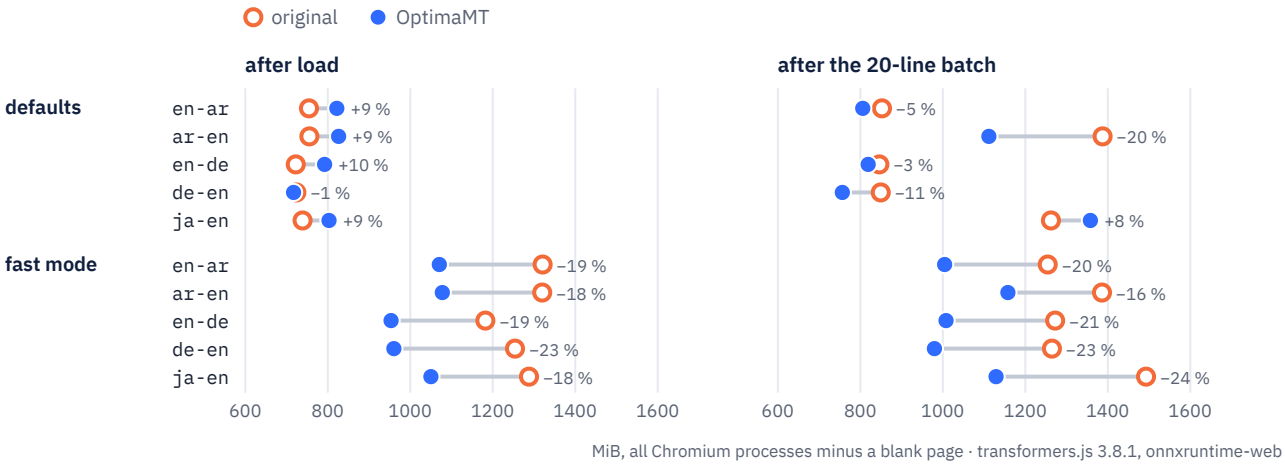


Figure 14. Browser memory after load and after a 20-line batch, original against OptimaMT (transformers.js 3.8.1, onnxruntime-web, Chromium 156).

Table 8. Browser results. Original → OptimaMT (results/browser.csv).

Pair	Settings	After load (MiB)	Change	After batch (MiB)	ms / token	Same tokens
en-ar	defaults	755 → 822	+9 %	852 → 806	39.1 → 38.7	yes
	fast mode	1321 → 1070	−19 %	1254 → 1004	29.3 → 29.2	yes
ar-en	defaults	755 → 826	+9 %	1388 → 1112	69.8 → 68.2	yes
	fast mode	1320 → 1078	−18 %	1386 → 1158	61.3 → 60.0	yes
en-de	defaults	722 → 792	+10 %	846 → 819	35.8 → 36.3	yes
	fast mode	1182 → 953	−19 %	1272 → 1008	25.4 → 25.8	yes
de-en	defaults	724 → 717	−1 %	850 → 756	33.7 → 33.1	yes
	fast mode	1254 → 960	−23 %	1265 → 980	25.3 → 26.4	yes
ja-en	defaults	739 → 803	+9 %	1262 → 1358	68.9 → 69.3	yes
	fast mode	1288 → 1050	−18 %	1492 → 1129	60.3 → 60.1	yes

In fast mode the saving holds: median 1288 → 1050 MiB after load (−18 %). With the defaults the rebuilt files are not lighter after loading (median 739 → 803 MiB). The WebAssembly heap only grows, so its size depends on the order of allocations as well as on what is kept; after the batch the rebuilt files use less memory in 4 of the five pairs. Fast mode is also faster in the browser, by 1.3× (median), less than in native ONNX Runtime.

6.8 Drop-in check

Table 9. Drop-in check: the published folders against the original files, in transformers.js (Node.js) and ONNX Runtime Python. Memory is Node.js resident memory after load, original → OptimaMT (results/dropin_transformersjs-*.json).

transformers.js	Pair	Same output (both settings)	Python same	RSS, defaults (MiB)	RSS, fast mode (MiB)
3.8.1	en-ar	yes	yes	466 → 400	736 → 612
3.8.1	de-en	yes	yes	446 → 397	692 → 578
3.8.1	ja-en	yes	yes	524 → 402	715 → 604
3.8.1	ar-de	yes	yes	495 → 416	754 → 632
4.3.2	en-ar	yes	yes	452 → 400	709 → 462
4.3.2	de-en	yes	yes	444 → 387	664 → 431
4.3.2	ja-en	yes	yes	448 → 390	698 → 454
4.3.2	ar-de	yes	yes	471 → 405	708 → 465

transformers.js 4.3.2 splits target-language tokens such as `>>ara<<` into pieces, for the original exports as well; 3.8.1 does not. transformers.js tokenizes with `tokenizer.json` and the Python script with `source.spm`, so their outputs can differ from each other (ja-en). They never differ between original and OptimaMT.

7 Using OptimaMT

Each pair is a complete Hugging Face repository, `OptimaMT/<source>-<target>`, in the same layout as the Xenova exports. Compared with the Xenova export it replaces, only `onnx/decoder_model_merged_quantized.onnx` differs (the five pairs exported from the Helsinki-NLP weights also carry generated tokenizer files, listed in the repository's NOTICE). Existing code switches by changing the model name. To get the memory benefit, enable fast mode.

7.1 JavaScript (transformers.js)

```
import { pipeline } from "@huggingface/transformers";

const translator = await pipeline("translation", "OptimaMT/de-en", {
  dtype: "q8",
  session_options: { extra: { session: { disable_quant_qdq: "1" } } }, // fast mode
});
const [out] = await translator("Der Leuchtturm steht an der Küste.");
console.log(out.translation_text); // The lighthouse is on the coast.
```

7.2 Python (ONNX Runtime, no PyTorch)

```
pip install onnxruntime sentencepiece numpy huggingface_hub
python scripts/translate.py OptimaMT/de-en "Der Leuchtturm steht an der Küste."
```

The script is about 100 lines and uses only ONNX Runtime, SentencePiece and NumPy. To enable fast mode in your own ONNX Runtime code, add one session option:

```
so = onnxruntime.SessionOptions()
so.add_session_config_entry("session.disable_quant_qdq", "1")
```

Four pairs (en-ar, en-zh, fr-ar, es-ar) expect a target-language token at the start of the input, for example `>>ara<<` or `>>cmn_Hans<<`. With transformers.js, use version 3.x for these pairs (Section 6.8).

7.3 Available pairs

The 49 directions cover Arabic, Chinese, Dutch, English, French, German, Hindi, Indonesian, Italian, Japanese, Korean, Polish, Russian, Spanish, Thai, Ukrainian and Vietnamese. Appendix A lists them. All are grouped in the OptimaMT collection on Hugging Face, and the results page (huggingface.co/spaces/OptimaMT/results) has the interactive charts and a live demo.

7.4 Used in production

OpenCircle (github.com/AmerAlreyahi/OpenCircle), a desktop screen translator for Windows and Linux, ships these models since version 0.1.4. It runs the decoders in fast mode on the CPU.

8 Limitations

- **CPU only.** All measurements use ONNX Runtime's CPU execution provider. GPU providers were not measured.
- **Greedy decoding only.** Beam search was not measured. It runs the same decoder, so the memory saving after load applies, but peak memory and speed under beam search were not checked.
- **One host.** Linux was measured directly and Windows in a virtual machine on the same CPU. Memory differs between the two (Windows reports the working set); the saving holds on both.
- **Browser defaults.** In the browser with ONNX Runtime's defaults the rebuilt files are not lighter right after loading (Section 6.7).
- **Version dependence.** The size of the saving depends on the ONNX Runtime version (Section 6.6). Token ids can also differ between ONNX Runtime versions, for both decoders alike.
- **Measurement noise.** Resident memory includes allocator effects of a few MiB.
- **Quality.** The translation quality is that of the original OPUS-MT models. The FLORES-200 scores show that the output is unchanged; they are not an evaluation of the models.

8.1 Future work

- Make the same change in the exporter, so that new merged decoders are built without the duplicate.
- Check other model families that combine a merged decoder with a tied, quantized output layer. They may have the same pattern; this has not been tested.
- Measure GPU execution providers and beam search.

9 Reproducibility, licences and acknowledgements

9.1 Reproducing the results

The repository github.com/AmerAlreyahi/OptimaMT contains the scripts, the pinned sources (URL, size and SHA-256 of every original file in `data/sources.json`), the test sentences and all raw results. The charts in this report are drawn from those result files.

Table 10. Scripts and what they produce.

Script	What it does	Output
convert.py	Rebuilds each decoder; checks it against the published SHA-256	rebuilt decoders
verify.py	Greedy decoding of fixed sentences, original vs rebuilt, both settings	identical.csv
measure.py	Memory and speed, one fresh process per configuration	reference.*csv, memory_all*.csv
inspect_graph.py	Saves ONNX Runtime's optimized graph and lists the large tensors	graph.json
flores.py	FLORES-200 devtest, every pair, both settings	flores.csv
tests/browser_test.py	transformers.js in headless Chromium	browser.csv
tests/dropin_test.py	Published folders against the originals	dropin_*.json

```
pip install -r requirements.txt
python scripts/convert.py en-ar   # byte-identical to the published decoder
python scripts/verify.py en-ar    # same token ids as the original, both settings
python scripts/measure.py         # memory and speed
```

9.2 Licences

The model weights are OPUS-MT by Jörg Tiedemann and the Language Technology Research Group at the University of Helsinki, licensed CC BY 4.0. The ONNX exports of 44 pairs are by Xenova (CC BY 4.0); five pairs were exported from the Helsinki-NLP weights in the same layout. The modified decoders are distributed under CC BY 4.0 with a notice describing the change. The scripts are MIT-licensed. FLORES-200 (CC BY-SA 4.0) is downloaded for scoring only and is not redistributed. This report is licensed CC BY 4.0. No endorsement by the original authors is implied.

9.3 Acknowledgements

This work builds entirely on other people's open work: OPUS-MT by Jörg Tiedemann, Santhosh Thottingal and the Helsinki-NLP group; the ONNX exports and transformers.js by Xenova (Joshua Lochner); Hugging Face Optimum; ONNX Runtime by Microsoft; FLORES-200 by the NLLB Team at Meta AI; SentencePiece; and sacrebleu.

References

1. J. Tiedemann and S. Thottingal. OPUS-MT – Building open translation services for the World. In Proceedings of the 22nd Annual Conference of the European Association for Machine Translation (EAMT), 2020.
2. M. Junczys-Dowmunt et al. Marian: Fast Neural Machine Translation in C++. In Proceedings of ACL 2018, System Demonstrations, 2018.
3. A. Vaswani et al. Attention Is All You Need. In Advances in Neural Information Processing Systems 30 (NeurIPS), 2017.
4. T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In Proceedings of EMNLP 2018, System Demonstrations, 2018.
5. NLLB Team et al. No Language Left Behind: Scaling Human-Centered Machine Translation. arXiv:2207.04672, 2022. (FLORES-200.)
6. ONNX Runtime, Microsoft. https://onnxruntime.ai
7. ONNX: Open Neural Network Exchange. https://onnx.ai
8. Hugging Face Optimum. https://github.com/huggingface/optimum
9. Transformers.js and the Xenova/opus-mt-* ONNX exports. https://github.com/huggingface/transformers.js, https://huggingface.co/Xenova
10. M. Post. A Call for Clarity in Reporting BLEU Scores. In Proceedings of the Third Conference on Machine Translation (WMT), 2018. (sacrebleu.)
11. M. Popović. chrF: character n-gram F-score for automatic MT evaluation. In Proceedings of the Tenth Workshop on Statistical Machine Translation (WMT), 2015.
12. K. Papineni, S. Roukos, T. Ward and W.-J. Zhu. BLEU: a Method for Automatic Evaluation of Machine Translation. In Proceedings of ACL 2002.

Appendix A All 49 pairs

Memory after loading encoder and decoder, original → OptimaMT, in MiB, ONNX Runtime 1.31.1. BLEU and chrF on FLORES-200 devtest (greedy); identical for both files and both settings in every pair (1,012 of 1,012 sentences).

Table 11. Every pair (models.json, results/memory_all*.csv, results/flores.csv). Fast = `disable_quant_qdq = 1`.

Pair	Vocabulary	Linux, fast	Linux, defaults	Windows, fast	Windows, defaults	BLEU	chrF
en-ar	62,802	527 → 282	282 → 220	504 → 256	198 → 166	22.03	54.14
en-fr	59,514	504 → 272	270 → 212	481 → 251	190 → 163	46.42	68.96
en-es	65,001	545 → 291	291 → 227	517 → 267	201 → 170	26.15	54.81
en-de	58,101	495 → 269	265 → 208	471 → 244	187 → 158	34.70	62.51
en-ru	62,518	525 → 281	281 → 220	500 → 257	195 → 167	24.14	52.27
en-zh	65,001	545 → 291	291 → 227	519 → 263	201 → 173	31.63	28.29
ar-en	62,834	529 → 284	283 → 222	504 → 258	196 → 167	33.43	60.37
fr-en	59,514	505 → 272	270 → 212	481 → 249	192 → 162	41.43	66.62
zh-en	65,001	544 → 290	290 → 226	518 → 266	202 → 171	22.34	52.63
ja-en	60,716	513 → 276	274 → 214	488 → 253	194 → 164	17.68	47.70
es-en	65,001	544 → 290	290 → 227	519 → 263	202 → 170	27.64	58.08
de-en	58,101	495 → 268	265 → 208	474 → 245	190 → 158	40.04	66.18
ru-en	62,518	527 → 282	281 → 220	502 → 259	196 → 165	28.51	57.03
it-en	80,379	648 → 333	255 → 216	626 → 311	231 → 195	29.60	59.75
nl-en	67,028	558 → 297	233 → 199	533 → 271	204 → 173	29.11	57.99
uk-en	61,587	520 → 279	278 → 218	495 → 257	195 → 164	27.11	55.26
pl-en	63,430	534 → 286	285 → 224	509 → 261	200 → 169	23.70	53.09
hi-en	61,127	518 → 278	277 → 217	491 → 256	194 → 164	18.11	44.76
id-en	54,796	472 → 258	254 → 201	449 → 235	181 → 155	32.89	60.75
vi-en	53,739	465 → 255	251 → 198	442 → 232	181 → 154	24.31	51.95
th-en	62,307	524 → 280	280 → 219	501 → 260	199 → 167	18.44	47.77
ko-en	65,001	545 → 291	291 → 228	520 → 265	200 → 169	19.22	48.81
en-it	80,035	645 → 333	255 → 216	621 → 307	230 → 190	27.32	57.45
en-nl	67,028	557 → 296	230 → 197	533 → 270	206 → 173	24.63	56.57
en-uk	61,587	519 → 279	277 → 217	494 → 256	195 → 163	18.66	47.35
en-hi	61,950	522 → 279	279 → 218	498 → 255	196 → 164	11.33	32.00
en-id	54,796	472 → 257	254 → 201	448 → 236	181 → 154	37.56	64.66
en-vi	53,685	464 → 257	251 → 198	441 → 232	179 → 155	24.50	43.68
de-es	61,301	516 → 278	277 → 217	493 → 254	193 → 165	20.88	49.83
de-fr	61,153	516 → 277	276 → 217	493 → 253	194 → 166	32.20	59.38
es-de	61,301	516 → 277	276 → 216	492 → 253	196 → 165	17.75	50.18
es-fr	74,822	610 → 318	244 → 208	586 → 295	222 → 185	24.74	55.42
es-it	59,031	501 → 270	269 → 211	479 → 249	192 → 163	20.58	51.41
es-ru	63,430	532 → 284	284 → 222	508 → 259	201 → 166	14.77	44.71
fr-de	61,153	516 → 276	276 → 216	494 → 255	195 → 164	25.22	55.56
fr-es	74,822	610 → 316	243 → 208	587 → 293	221 → 184	21.37	49.90
fr-ru	64,377	539 → 287	287 → 224	513 → 264	200 → 168	15.07	44.17
it-es	59,031	501 → 271	269 → 211	478 → 246	190 → 161	21.72	51.17
it-fr	59,351	503 → 272	270 → 212	481 → 249	191 → 162	30.22	58.86
ru-es	63,430	533 → 285	284 → 222	509 → 260	197 → 168	18.07	46.65

ru-fr	64,377	540 → 288	287 → 224	515 → 262	199 → 168	24.43	52.95
ru-uk	57,777	494 → 268	265 → 208	470 → 244	189 → 159	20.36	49.51
uk-ru	57,777	494 → 268	265 → 208	471 → 243	188 → 158	23.28	51.04
nl-fr	57,642	494 → 270	266 → 210	467 → 242	187 → 158	22.88	52.15
fr-ar	62,419	525 → 281	281 → 220	500 → 257	196 → 165	13.61	45.82
es-ar	62,518	526 → 282	281 → 220	502 → 260	197 → 166	10.52	43.09
ar-fr	64,505	542 → 290	288 → 226	514 → 265	200 → 170	26.01	53.48
ar-es	62,526	526 → 281	281 → 220	503 → 256	196 → 168	17.85	46.35
ar-de	63,079	530 → 283	283 → 221	506 → 259	199 → 167	17.43	47.48

Appendix B Glossary

Table 12. Terms used in this report.

Term	Meaning
Attention	A layer that lets each token take a weighted average of other tokens' vectors, with weights computed from how relevant they are.
BLEU, chrF	Automatic translation scores that compare system output with reference translations, by word n-grams (BLEU) or character n-grams (chrF).
Constant folding	Computing operations whose inputs are all constants once, at load time, and storing the result.
Cross-attention	Attention from the decoder to the encoder's output, i.e. from the translation to the source.
DequantizeLinear	The ONNX operation that converts 8-bit integers back to float: (q − zero point) × scale.
Embedding	The vector of numbers (512 here) that represents one vocabulary entry.
Encoder, decoder	The two halves of the model: the encoder reads the source, the decoder writes the translation token by token.
Fast mode	In this report: ONNX Runtime with the session option `session.disable_quant_qdq = 1`.
Greedy decoding	Choosing the highest-scoring token at every step.
If node	An ONNX operation that runs one of two subgraphs depending on a boolean input.
Key/value cache	Attention keys and values of earlier positions, kept between decoder steps.
MiB	Mebibyte, 2^{20} bytes (1,048,576 bytes).
ONNX, ONNX Runtime	A file format for neural network graphs, and Microsoft's engine for running them.
Output layer	The final matrix multiplication that turns a 512-number vector into one score per vocabulary entry. In OPUS-MT it reuses the embedding table.
Quantization	Storing weights as 8-bit integers with a scale and zero point instead of 32-bit floats.
RSS, working set	The memory of a process that is resident in physical RAM, as reported by Linux and Windows.
SentencePiece	The tokenizer used by OPUS-MT; it splits text into word pieces learned from data.
Token	A word or word piece; the unit the model reads and writes.
Vocabulary	The fixed list of tokens a model knows; its size differs per language pair.

About the author

Amer Alreyahi is an AI & PPM Solution Architect based in Riyadh, Saudi Arabia. He holds a B.Sc. in Computer Science (AI & Data Science) from Tafila Technical University (2024) and is a Broadcom Certified Partner (Clarity PPM). He builds OpenCircle, an open-source desktop screen translator, and maintains the OptimaMT models.

Web	ameralreyahi.com
Email	amerzuher@outlook.com
GitHub	github.com/AmerAlreyahi
LinkedIn	linkedin.com/in/ameralreyahi
Hugging Face	huggingface.co/AmerAlreyahi
ResearchGate	researchgate.net/profile/Amer-Alreyahi

Project links. Models and collection: huggingface.co/OptimaMT. Code, data and this report's sources: github.com/AmerAlreyahi/OptimaMT. Interactive results and live demo: huggingface.co/spaces/OptimaMT/results.

How to cite. Alreyahi, A. (2026). OptimaMT: Lighter OPUS-MT Translation Models for ONNX Runtime. Technical Report v1.0. https://github.com/AmerAlreyahi/OptimaMT